

# OpenUML

## Guide to Creating an IDEF0 Diagram

*Open source project*

**Author:** Ahhmed Jalife

November 5, 2026

For OpenUML 0.1.9

# Table of Contents


**1. Introduction** ..... **2**

**2. The OpenUML Interface** ..... **2**

**3. Creating an IDEF0 Diagram** ..... **3**

3.1 Deleting the Default Class Diagram ..... 3

3.2 Adding an IDEF0 Diagram ..... 3

3.3 The Untitled File ..... 3

3.4 IDEF0 Model 1 ..... 4

3.5 A-0 Context Diagram ..... 4

3.6 A0 Top Function ..... 5

**4. Decomposing the A0 Function** ..... **6**

4.1 Starting the Decomposition ..... 6

4.2 Result of the Decomposition ..... 6

4.3 Decomposing a Function in More Detail ..... 7

4.4 Changing the Number of Boxes ..... 7

4.5 Moving Between Diagrams ..... 8

**5. Toolbox Elements** ..... **8**

5.1 Function ..... 8

5.2 Arrow (ICOM Arrow) ..... 8

5.3 More Uses of Arrow ..... 9

5.4 Call Arrow ..... 9

5.5 Model Note ..... 9

5.6 Annotations ..... 9

**6. Connecting the Arrows of the Parent Diagram** ..... **10**

6.1 Pending Arrows ..... 10

6.2 Box Table ..... 10

6.3 Advanced Arrows Panel ..... 11

6.4 Tunneled Arrows ..... 12

**7. Adjusting Arrows and the Sheet** ..... **13**

7.1 Arrow Routes ..... 13

7.2 Colors ..... 13

7.3 Sheet and Title Block ..... 13

7.4 Dark Canvas ..... 14

**8. Reviewing the Model** ..... **14**

**9. Saving and Exporting** ..... **15**

9.1 Saving ..... 15

9.2 Exporting Diagrams ..... 15

9.3 Interactive Website ..... 15

**10. Rearranging the Layout** ..... **17**

# 1. Introduction

OpenUML is an open source UML diagram editor. This guide explains, step by step, how to create an IDEF0 diagram in OpenUML: from getting to know the interface to decomposing functions, using the toolbox elements, connecting the arrows of the parent diagram, reviewing the model and exporting it.

This edition is for OpenUML 0.1.9.

# 2. The OpenUML Interface

**Model explorer:** where the elements you place on the canvas are listed. The path to the selected row is highlighted.

**Working Diagrams:** the list of open diagrams, which also appear as tabs above the canvas. Opening a file lists all its diagrams.

**Toolbox:** the elements available for use; these change depending on the selected diagram. OpenUML automatically starts with a class diagram.

**Styles, Properties and Documentation:** the format, the data and the documentation of whatever you have selected.

**Status bar:** the bottom strip shows whether your work is saved and, in IDEF0, how many warnings and pending arrows the diagram has.

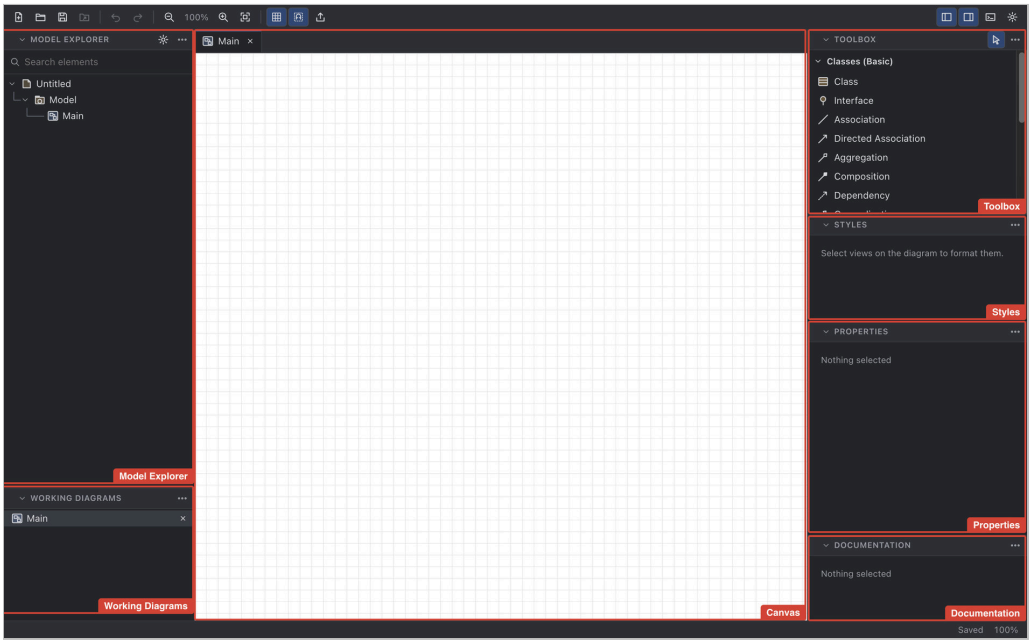



**Figure 1.** OpenUML main window with its panels.

# 3. Creating an IDEF0 Diagram

## 3.1 Deleting the Default Class Diagram

To delete the class diagram, right-click **Model** and select **Delete from Model**.

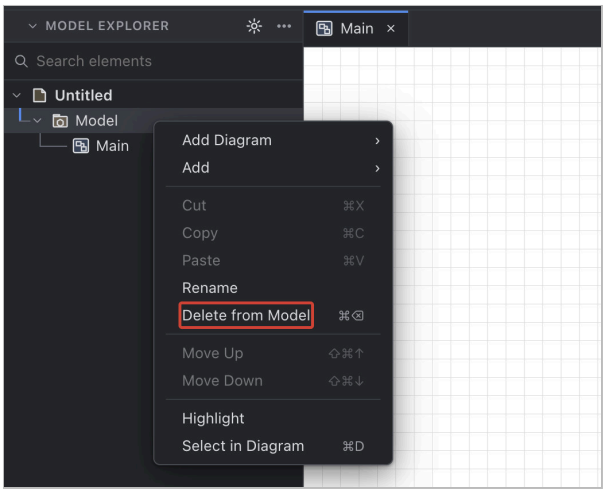



**Figure 2.** Context menu of Model with the Delete from Model option.

## 3.2 Adding an IDEF0 Diagram

To start our IDEF0 diagram, right-click **Untitled** and choose **Add Diagram > IDEF0 Diagram**. The same option is in the top menu: **Model > Add Diagram > IDEF0 Diagram**.

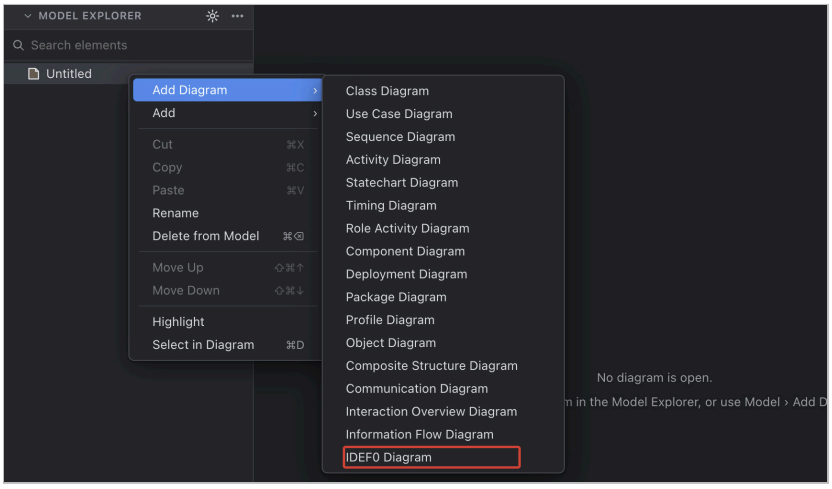



**Figure 3.** Right-click Untitled > Add Diagram > IDEF0 Diagram.

## 3.3 The Untitled File

As we can see, there are 3 default elements inside Untitled. **Untitled** is the file we are working on, that is, what gets saved as an .mdj file. When saving, we can choose the name, or we can also right-click and select **Rename**.

## 3.4 IDEF0 Model 1

Next we have **IDEF0 Model 1**, which is the complete IDEF0 diagram, because a single file can hold several different diagrams at the same time. We could have IDEF0 Model 1 and IDEF0 Model 2, and these would be two different diagrams.

When IDEF0 Model 1 is selected, Properties lets us write the **Purpose** and **Viewpoint** of the model, which are shown on the A-0 diagram.

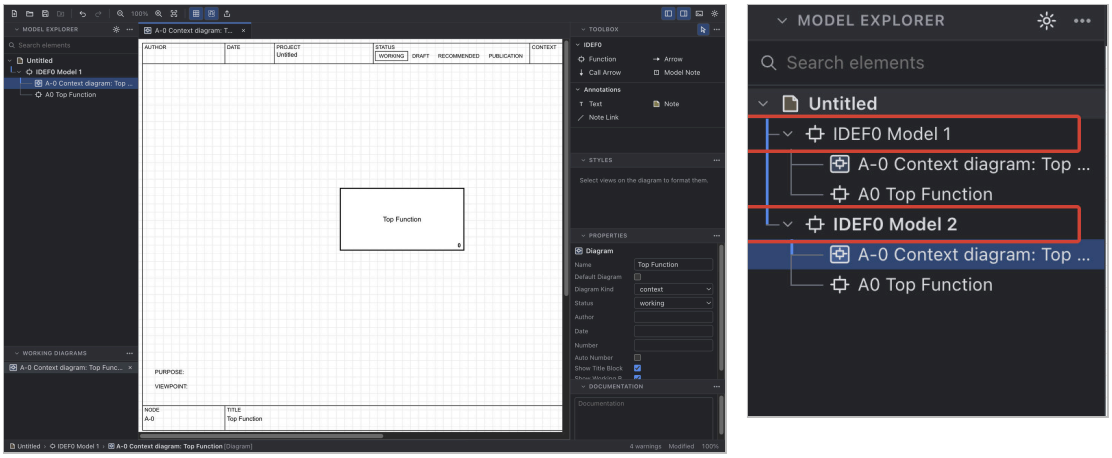



**Figure 4.** Canvas with the Model explorer (left) and several IDEF0 models inside the same file (right).

## 3.5 A-0 Context Diagram

Then comes the **A-0 Context Diagram**. When we click it, the whole frame inside the canvas becomes selected; this is where we can edit the attributes highlighted in red.

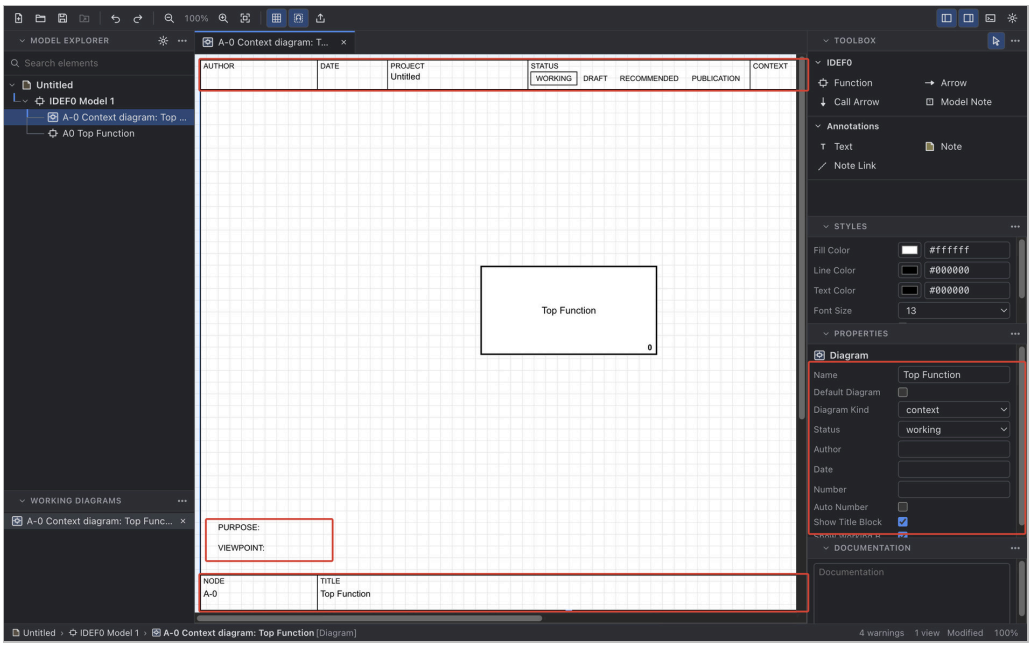



**Figure 5.** A-0 Context Diagram with its editable attributes highlighted in red.

## 3.6 A0 Top Function

The next element is **A0 Top Function**, the box inside the A-0 Context Diagram. By double-clicking the text inside the central box we can change its name, as well as in the properties section on the right. As the name suggests, this is our A0, where we start by naming our process.

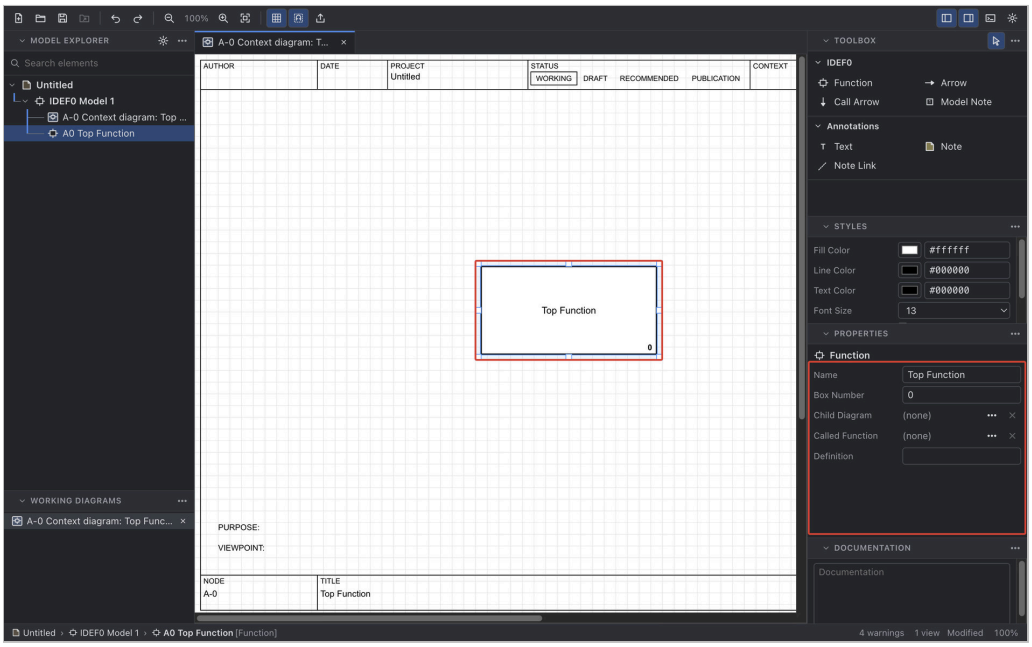



**Figure 6.** A0 Top Function selected, with its properties section.

# 4. Decomposing the A0 Function

## 4.1 Starting the Decomposition

There are two ways to decompose A0: right-click the box and choose **Add Diagram > IDEF0 Diagram** or **IDEF0 > Decompose...** (the latter is also in the top menu, under **Model > IDEF0**).

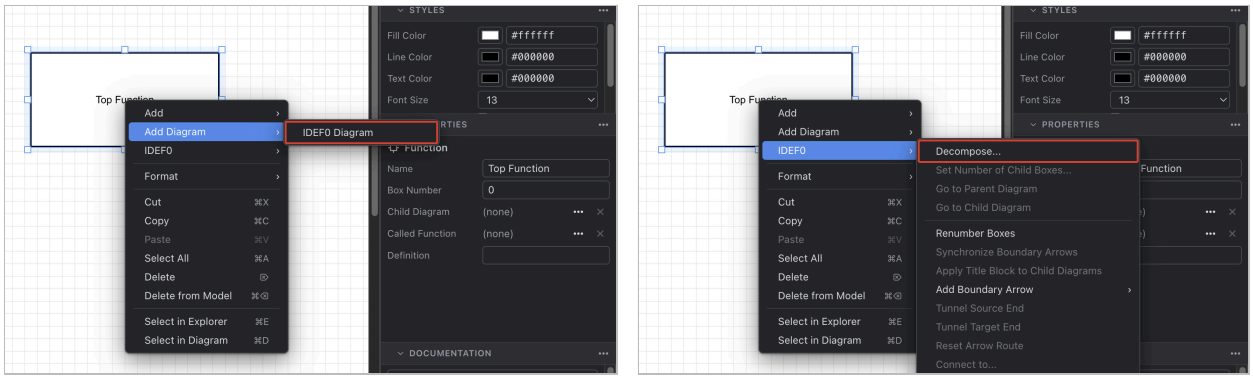



**Figure 7.** Two ways to decompose: Add Diagram > IDEF0 Diagram (left) and IDEF0 > Decompose... (right).

Both lead to this screen, where we enter the number of boxes we want, from 1 to 6; IDEF0 recommends 3 to 6 (we can change this number later, as explained in section 4.4).

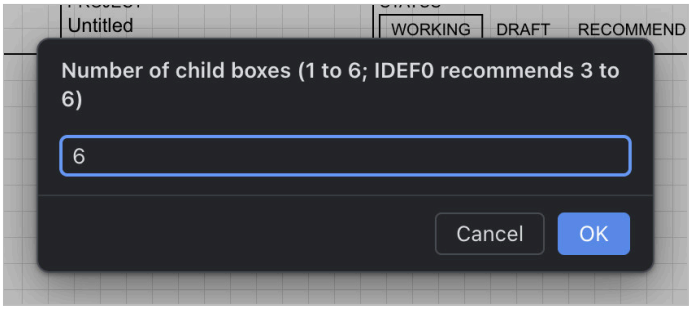



**Figure 8.** Dialog for entering the number of child boxes.

## 4.2 Result of the Decomposition

For this example I created 6 boxes and, as we can see, there are 7 new elements below A0 Top Function. First we have **A0 Diagram**, where we see the 6 boxes; do not confuse it with A0 Top Function, where we only see A0. After A0 Diagram come the 6 elements we created (all of this inside the Model explorer). That is where we can rename the functions, just as by double-clicking the box on the canvas.

If A0 already had arrows, they do not connect to the new boxes by themselves: they wait on the edge of the sheet until you decide which box uses each one (section 6).

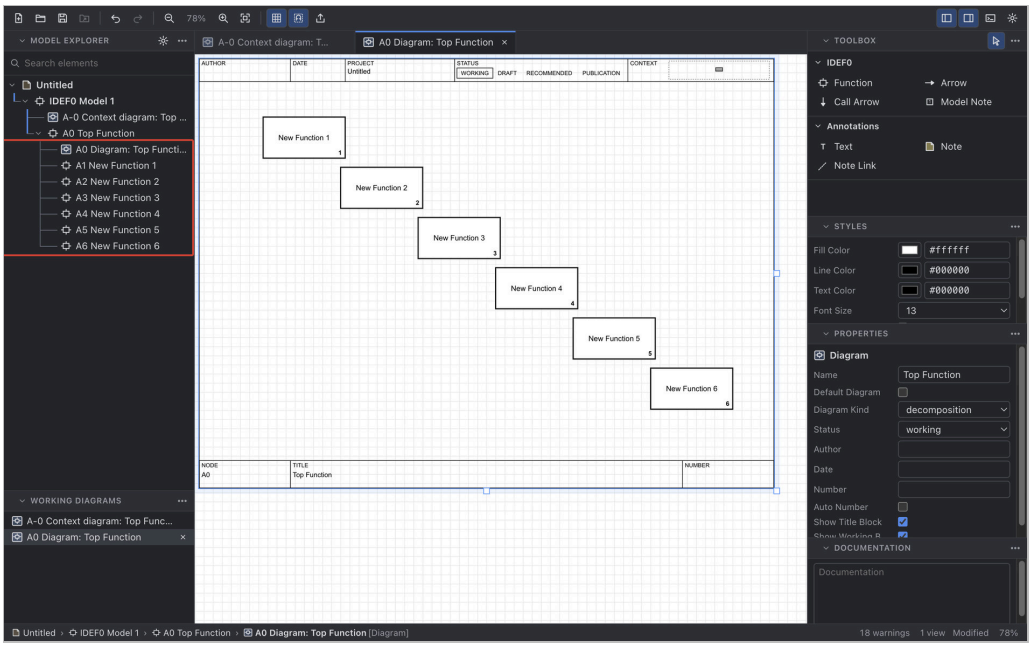



**Figure 9.** A0 decomposed into 6 functions and their elements in the Model explorer.

## 4.3 Decomposing a Function in More Detail

If, after finishing the 6 boxes, we realize that one box can be explained further, we can repeat the Decompose step or Add Diagram > IDEF0 Diagram, first selecting the box we want to expand.

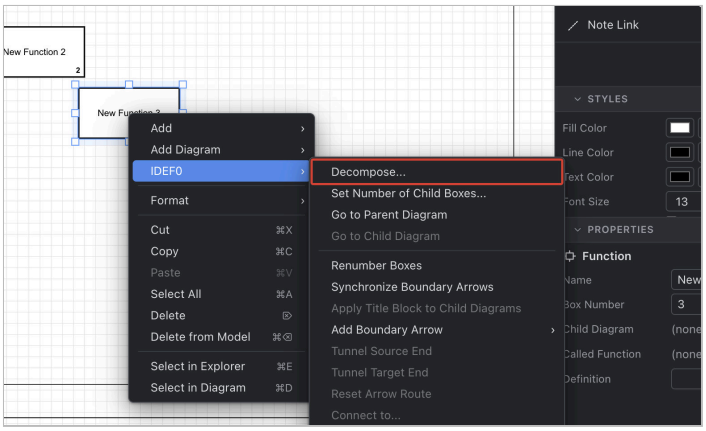



**Figure 10.** Options to decompose the selected function.

## 4.4 Changing the Number of Boxes

With the diagram open, enter the new number in **Model > IDEF0 > Set Number of Child Boxes...**. If it is higher, boxes are added. If it is lower, OpenUML asks you to tick which ones to remove: each removed box takes its arrows and sub-diagrams with it, the rest are renumbered, and everything can be undone in one step.

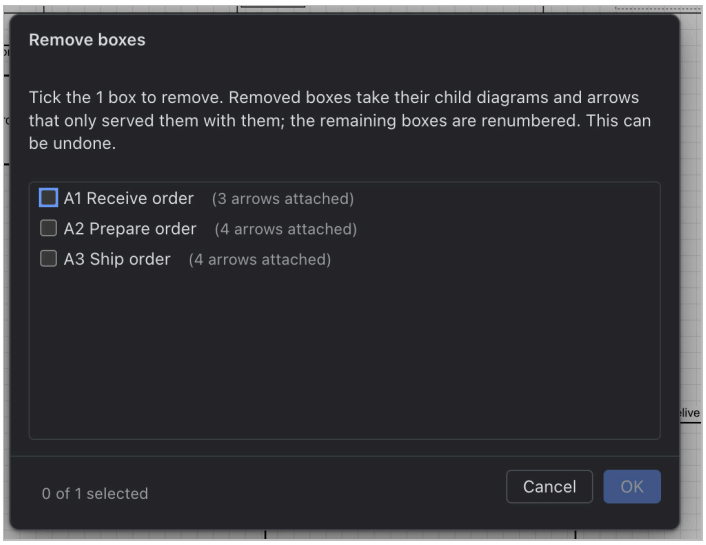



**Figure 11.** Remove boxes dialog, for choosing which boxes to remove.

### 4.5 Moving Between Diagrams

To go up or down a level, use **Model > IDEF0 > Go to Parent Diagram** and **Go to Child Diagram**. You can also open a diagram by double-clicking it in the Model explorer, or switch between diagrams with the tabs and Working Diagrams.

## 5. Toolbox Elements

The following sections explain how to use each of the toolbox elements.

### 5.1 Function

The box for an activity or process. Click or drag on the diagram to create it. Give it a name that starts with a verb, for example “Review order”. The box number (A1, A2...) is assigned automatically.

### 5.2 Arrow (ICOM Arrow)

This is the ICOM arrow. Its role depends on the side of the box it arrives at:

- **Left:** input (Input), what gets transformed.
- **Top:** control (Control), the rules or standards.
- **Bottom:** mechanism (Mechanism), who or what performs it.
- **Right:** output (Output), the result.

To use Arrow, simply press outside the box, on the side of the box where you want the arrow to arrive. That is, if you want it to be an output, select the box and drag to the right; if you want it to be an input, drag from the left, from outside the box into the box, and it will be an input. If you release the arrow inside the box, the side the pointer came in through is the one that counts.

In the Model explorer, the arrows of each box are grouped in the Inputs, Controls, Outputs and Mechanisms folders.

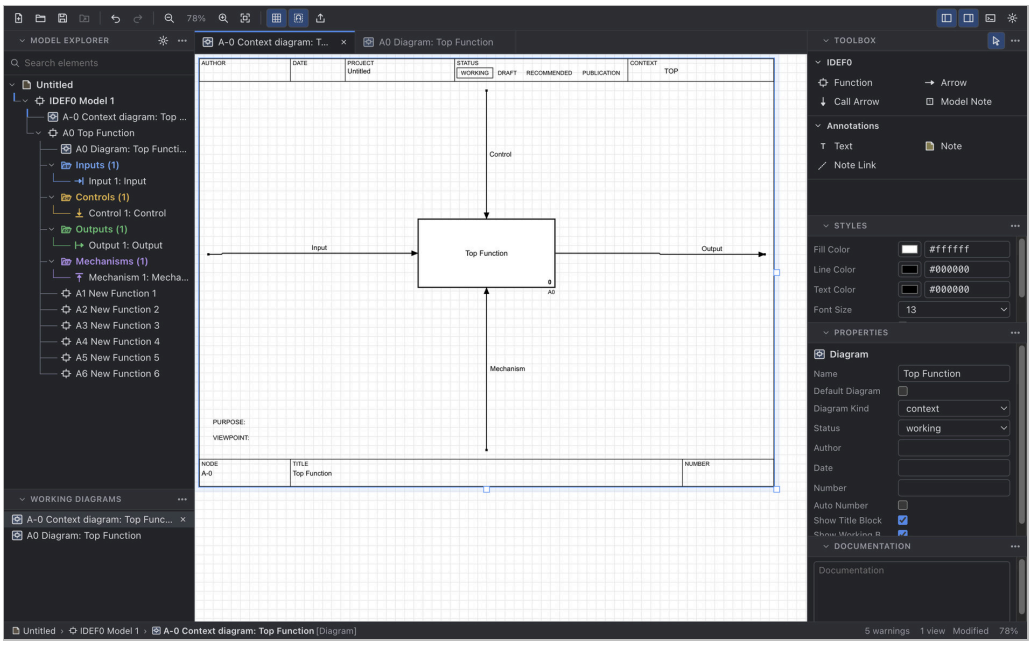



**Figure 12.** The four ICOM arrows on a function and their folders in the Model explorer.

## 5.3 More Uses of Arrow

- **From the edge of the sheet:** creates a boundary arrow, which is also added to the parent diagram. It is also available in **Model > IDEF0 > Add Boundary Arrow**.
- **From an existing arrow:** creates a branch (fork).
- **Into an existing arrow:** creates a join.

## 5.4 Call Arrow

Links a box to another box that performs that function, that is, it “calls” it from another part of the model. Each box can have only one. Its label shows the reference of the called box.

## 5.5 Model Note

A numbered note of the model. In the diagram only the number is visible, and the text is written when editing it. It is used for formal clarifications of the model.

## 5.6 Annotations

These are common to all diagrams:

- **Text:** free text over the diagram, with no meaning in the model.
- **Note:** a note with text, like a sticky comment.
- **Note Link:** links a Note to the element it comments on.

# 6. Connecting the Arrows of the Parent Diagram

The examples in this section use the process “Fulfill an order”, with three boxes on its A0 diagram.

## 6.1 Pending Arrows

When you decompose a box, its arrows do not connect by themselves: the modeler decides which box uses each one. Meanwhile, each parent arrow waits on the edge of the sheet as a **pending arrow**: a dashed line with a circle, in the color of its role, with its ICOM code (I1, C1, O1, M1) and its name. The status bar counts the ones left.

To connect a pending arrow, drag its circle to the side of the box that matches its role. Dragging it to more boxes creates a fork (a join, for outputs).

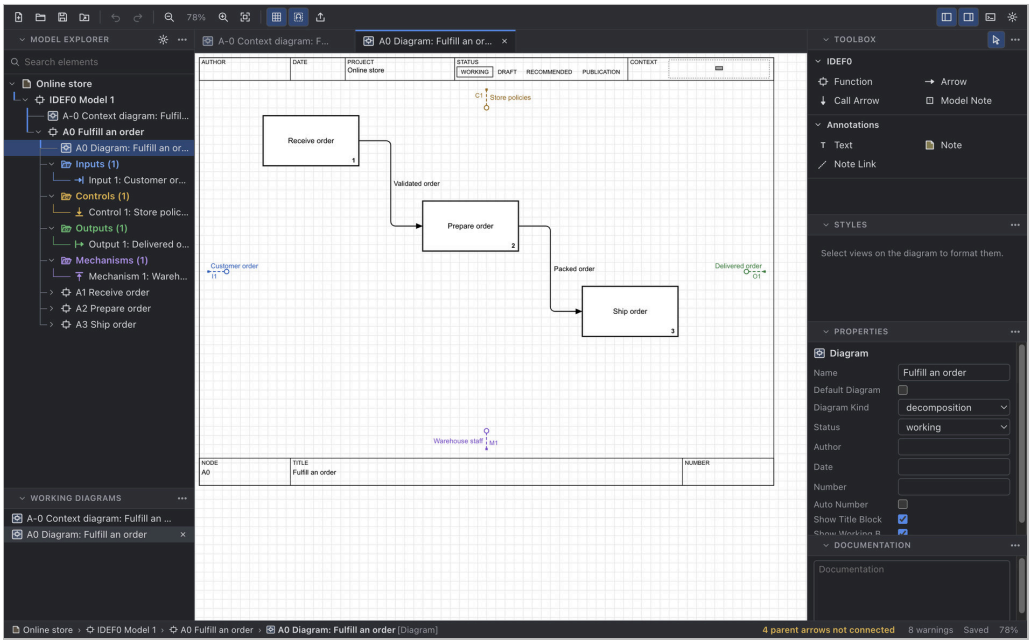



**Figure 13.** Pending arrows on A0 and their count in the status bar.

## 6.2 Box Table

You can also click a pending arrow, or choose **Connect to...** on a boundary arrow, to open a table of the boxes on the diagram. The boxes you tick are previewed on the canvas with dashed lines, and the table shows the result (1 box, or a fork to several). **Connect** applies everything in a single undo step. If a box already has its own child diagram, OpenUML warns you before taking an arrow away from it.

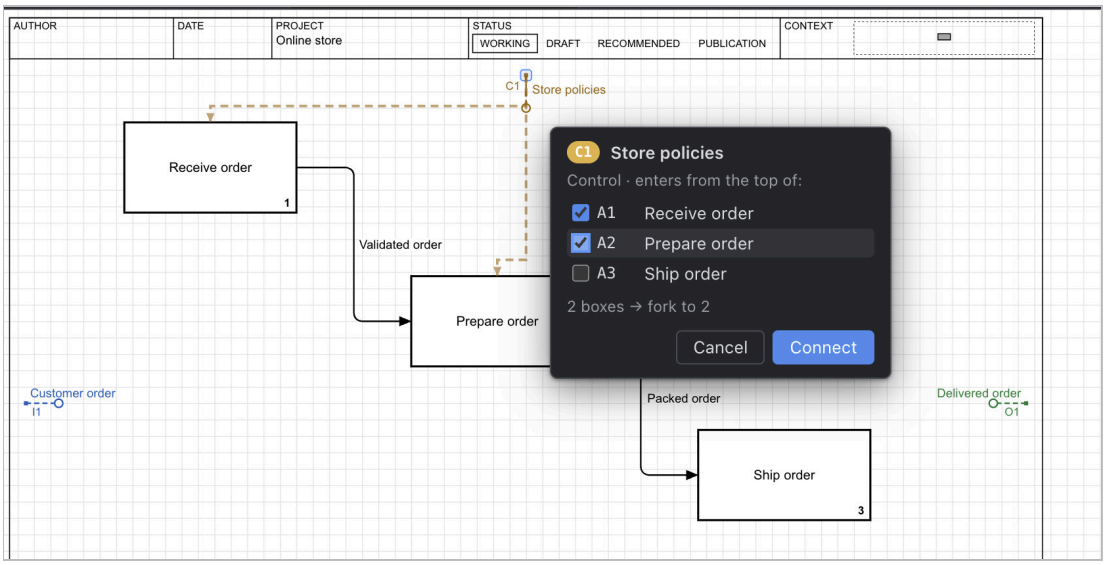



**Figure 14.** Box table of a control arrow, with the preview on the canvas.

## 6.3 Advanced Arrows Panel

To distribute several arrows at once, right-click a decomposition diagram in the Model explorer and choose **Advanced Arrows** (also in **View > Panels**). The panel shows one row per parent arrow and one column per box; each checkbox applies at once. It is also useful for redistributing the arrows of older files.

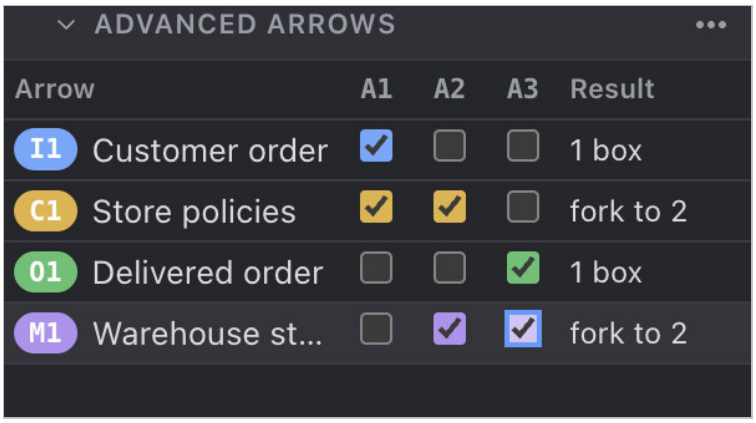



**Figure 15.** Advanced Arrows panel: parent arrows by box.

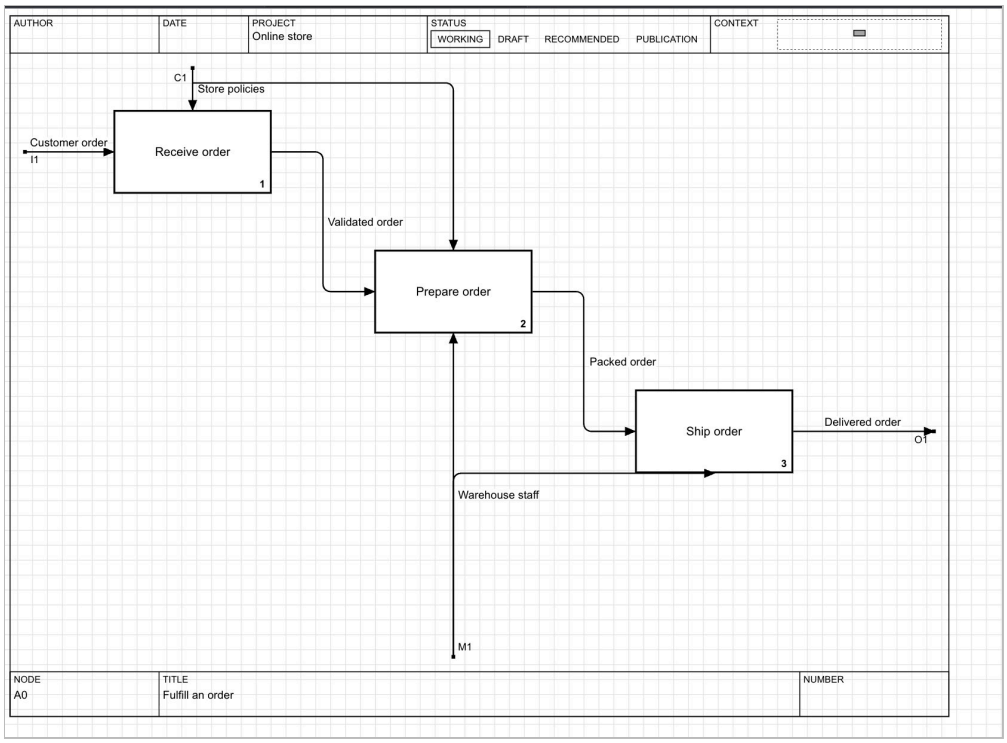



**Figure 16.** A0 diagram with all the parent arrows connected.

## 6.4 Tunneled Arrows

A tunneled arrow is one that must not appear on the other level of the model. With the arrow selected, choose **Model > IDEF0 > Tunnel Source End** or **Tunnel Target End**. For example, a mechanism drawn from the edge of A0 is also added to the parent diagram; with Tunnel Source End it stops appearing there and stays only on A0. Tunneling at a decomposed box also removes that arrow from its child diagram. The same option is in Properties (Tunnel Source, Tunnel Target).

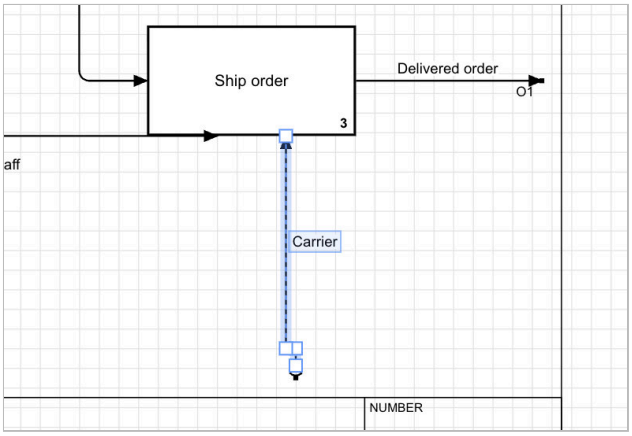



**Figure 17.** Mechanism arrow with its source end tunneled.

# 7. Adjusting Arrows and the Sheet

## 7.1 Arrow Routes

With the selection tool, select an arrow and drag one of its segments: horizontal ones move up or down and vertical ones move sideways. You can also drag an end to another side of the box or to another box. **Model > IDEF0 > Reset Arrow Route** restores the automatic route.

## 7.2 Colors

In the Styles panel, **Line Color** and **Text Color** change the color of an arrow. A fork is treated as a single arrow, and the color is kept on the parent and child diagrams. To color the whole model by the role of each arrow, turn on **Format > IDEF0 > Color Arrows by Role**.

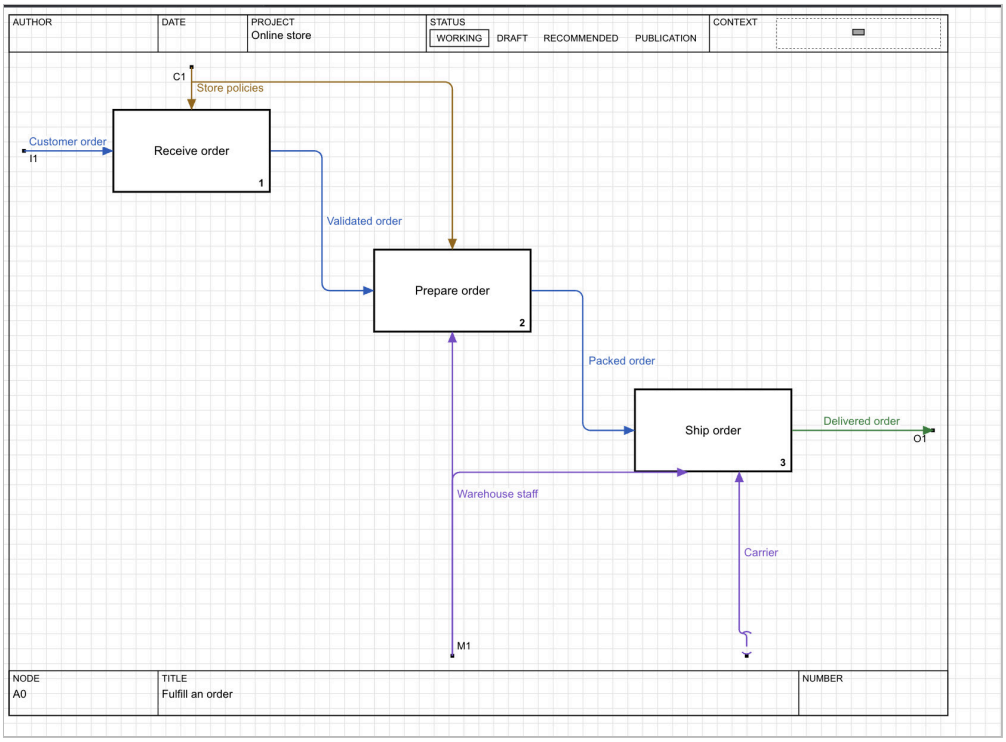



**Figure 18.** A0 diagram with Color Arrows by Role turned on.

## 7.3 Sheet and Title Block

In **Format > IDEF0 > Sheet Size** you choose the size of the sheet (Letter, Legal, Tabloid/Ledger, A4 or A3), or **Fit to Content** to fit it to the content; **Apply Sheet Size to Child Diagrams** applies it to the child diagrams. You can also select the frame and drag its handles.

The status of the diagram (Working, Draft, Recommended or Publication) is changed in **Format > IDEF0** or in Properties. A new child diagram copies the author, date and status of its parent and takes the next sheet number; to repeat this later, use **Model > IDEF0 > Apply Title Block to Child Diagrams**.

## 7.4 Dark Canvas

**View > Dark Canvas** (Cmd/Ctrl+Shift+D) draws diagrams on a dark canvas; in **Preferences** you can also choose Light, Dark or Follow App. It only changes how the diagram looks: the .mdj file stays the same and exports stay light, unless you turn on **Change default export color to dark** in Preferences.

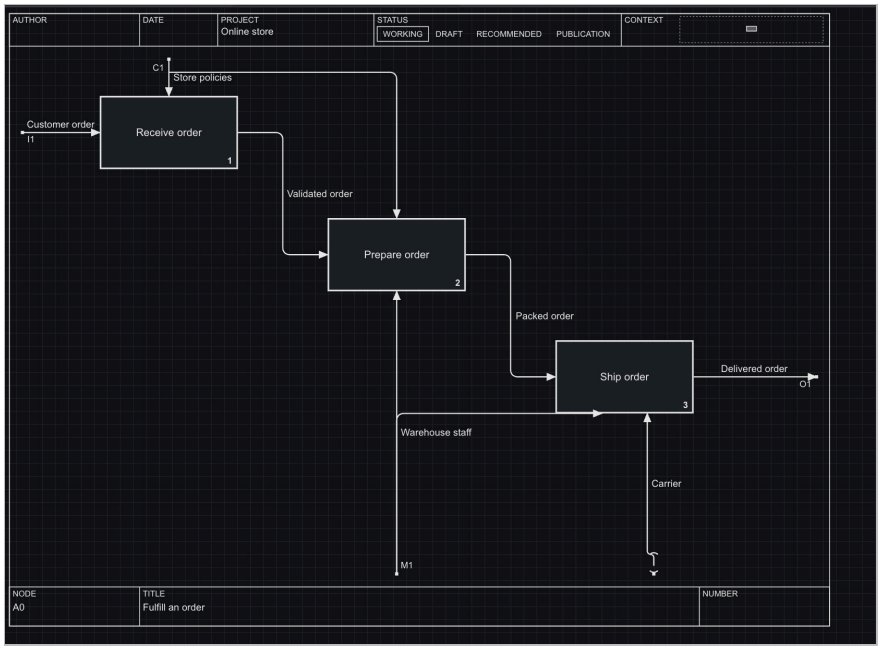



**Figure 19.** A0 diagram on the dark canvas.

# 8. Reviewing the Model

OpenUML reviews the model as you work. Warnings never prevent editing: the status bar shows how many there are and clicking the count opens the list. You can also open it with **Model > IDEF0 > Validate IDEF0 Model**. It points out, for example, boxes with no control or no output, names that do not look like verbs, pending arrows or a missing Purpose and Viewpoint.

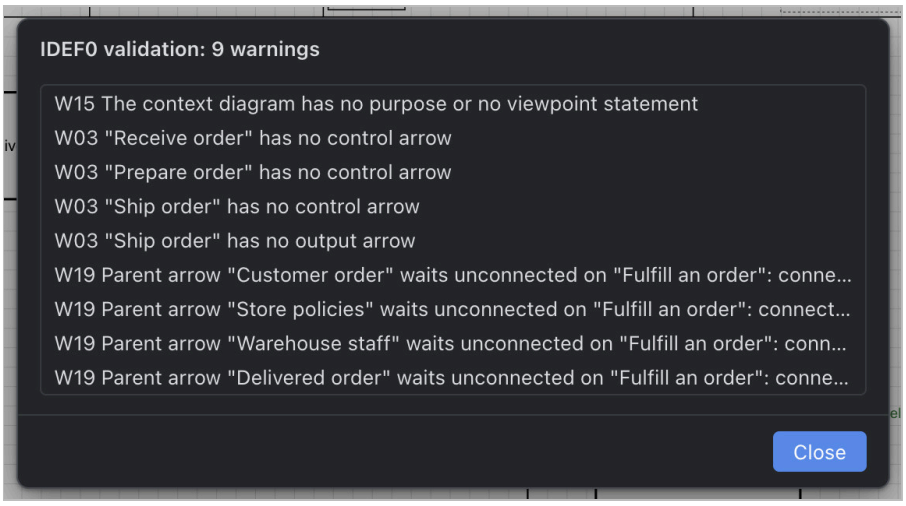



**Figure 20.** List of model warnings.

Other useful options are **Show Node Index**, which shows the index of the boxes of the model, and **Renumber Boxes**, which numbers the boxes of a diagram again.

# 9. Saving and Exporting

## 9.1 Saving

**File > Save** saves the project as an .mdj file. Once it has a file, it saves by itself every minute if there are changes; the interval is set in **Preferences > Files**. If OpenUML closes unexpectedly, it offers to recover the unsaved changes the next time it opens.

To see a finished model, open the bundled example from **File > Open Example**.

## 9.2 Exporting Diagrams

**File > Export Diagram As** exports the current diagram as **PNG**, **SVG** or **PDF**, or every diagram with **All to PNGs**, **All to SVGs** and **All to One PDF**. The last one creates a single file with one page per diagram; sheets with a paper size print at that size.

For the most common case, the **Export** button in the toolbar offers **PNG** or **PDF** of the current diagram.

## 9.3 Interactive Website

**File > Export Interactive Website...** creates one HTML file that anyone can open in a browser, even without internet. When exporting you choose the title and the diagrams.

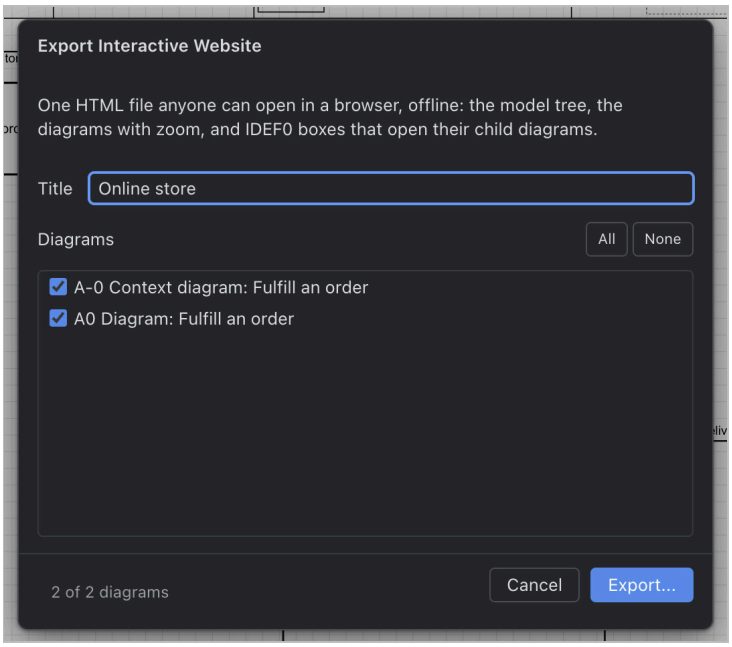



**Figure 21.** Export Interactive Website dialog.

The page includes the model tree and the diagrams with zoom, and clicking a decomposed box enters its child diagram. The left and right arrow keys walk the model box by box; below the diagram you see the inputs, controls, outputs and mechanisms of the current box.

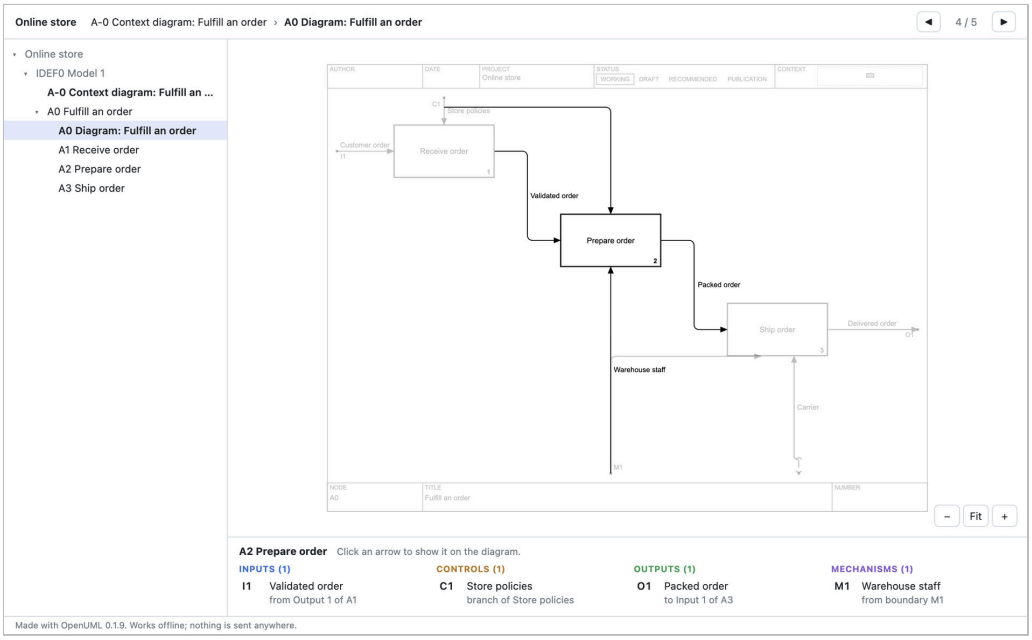



**Figure 22.** Exported website, on box A2.

# 10. Rearranging the Layout

If you want to move the Toolbox or other panels to a different place, you can drag them by pressing the 6 dots that appear when you hover over the panel title. You can also press and hold the title for 0.4 seconds, and the panel will pop out of its place so you can drag it wherever you want. The layout is remembered; to go back to the original, use **View > Reset Layout**.




**Figure 23.** The 6 dots in the title let you drag the panel.

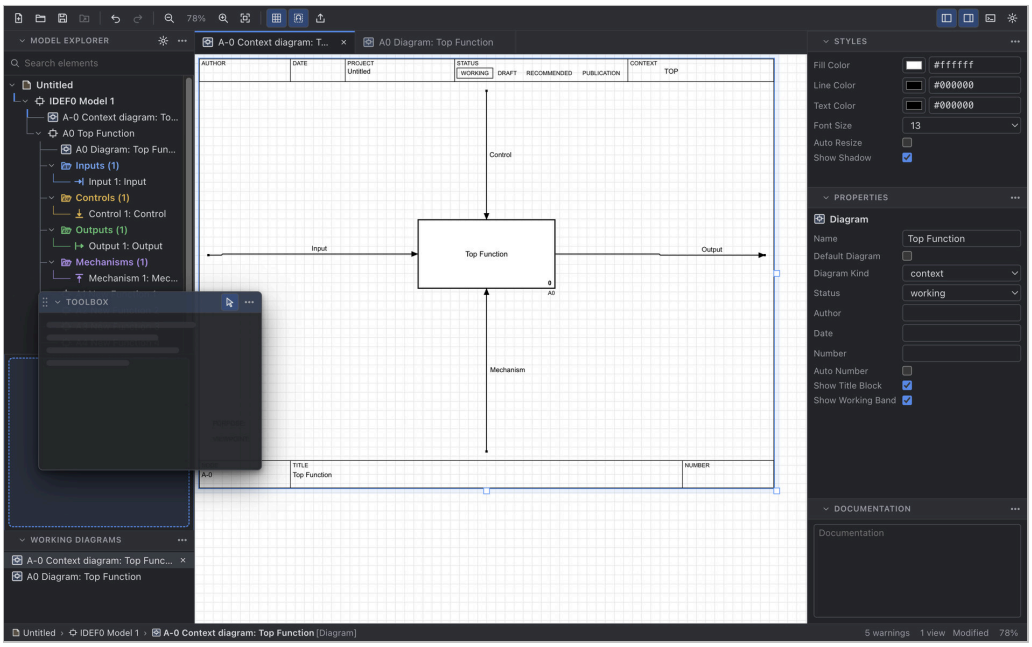



**Figure 24.** Moving a panel: the Toolbox being dragged to the left side.